TASK 9: Implement and Test the Fare Logic

Name: Nagineni Sunil Kumar

Register Number: 192472330

1. Aim

To validate the fare calculation using normal journeys, peak-hour boundaries, minimum-balance cases, and invalid inputs.

2. C++ Program

```
#include <iostream>

using namespace std;

enum class Zone { Inner, Middle, Outer };

long long baseFare(Zone z) {

   switch (z) {

      case Zone::Inner: return 1000;

      case Zone::Middle: return 2000;

      case Zone::Outer: return 3000;    }

   return -1;}

bool isPeak(int hour) {

   return (hour >= 8 && hour < 10) ||

         (hour >= 17 && hour < 20); }

long long calculateFare(Zone z, int hour) {

   if (hour < 0 || hour > 23)

      return -1;
```

```
    long long fare = baseFare(z);
    if (fare < 0)
        return -1;
    if (isPeak(hour))
        fare = fare * 120 / 100;
    return fare;}
bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= 5000;}
int main() {
    cout << "Inner off-peak: "
        << calculateFare(Zone::Inner, 12) << " paise\n";
    cout << "Middle peak: "
        << calculateFare(Zone::Middle, 9) << " paise\n";
    cout << "Outer peak: "
        << calculateFare(Zone::Outer, 18) << " paise\n";
    cout << "At 10:00: "
        << calculateFare(Zone::Inner, 10) << " paise\n";
    cout << "At 20:00: "
        << calculateFare(Zone::Inner, 20) << " paise\n";
    cout << "Balance 6000, fare 1000: "
        << (canTravel(6000, 1000) ? "Allowed" : "Denied")
        << '\n';
```

```
    cout << "Balance 5999, fare 1000: "
         << (canTravel(5999, 1000) ? "Allowed" : "Denied")
         << '\n'; }
```

3. Expected Output

Inner off-peak: 1000 paise

Middle peak: 2400 paise

Outer peak: 3600 paise

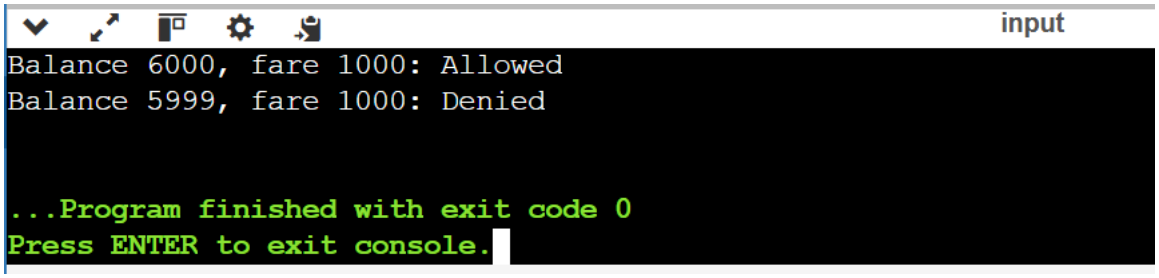
At 10:00: 1000 paise

At 20:00: 1000 paise

Balance 6000, fare 1000: Allowed

Balance 5999, fare 1000: Denied

Obtained output:-



4. Test Table

Test	Input	Expected Result
1	Inner, 12:00	₹10.00
2	Middle, 09:00	₹24.00
3	Outer, 18:00	₹36.00
4	Inner, 08:00	₹12.00

5	Inner, 10:00	₹10.00
6	Inner, 17:00	₹12.00
7	Inner, 20:00	₹10.00
8	Balance ₹60, fare ₹10	Allowed; remaining ₹50
9	Balance ₹59.99, fare ₹10	Denied
10	Hour 25	Invalid hour
11	Negative recharge	Reject recharge
12	Letters entered for hour	Show input error and request valid input

Note: The program above demonstrates fare calculation and balance validation. For the final submission, add input validation for the zone and hour, reject negative recharge amounts, and test invalid text input as required by the assignment.

5. Five-Journey Calculation

For this example, use an initial balance of ₹200 and these journeys:

Journey	**Fare**	**Balance After Journey**
Inner, off-peak	₹10	₹190
Middle, peak	₹24	₹166
Outer, off-peak	₹30	₹136
Inner, peak	₹12	₹124
Middle, off-peak	₹20	₹104

Total fare = ₹10 + ₹24 + ₹30 + ₹12 + ₹20 = **₹96**.

Final balance = ₹200 − ₹96 = **₹104**.

All five journeys satisfy the minimum-balance rule.

6. Alternative Concept and Complexity

Instead of separate checks, store test inputs and expected outputs in a test table and loop through it. For (t) test cases, the time complexity is **O(t)** and the test table uses **O(t)** space. New boundary cases can be added by inserting new rows.

7. Conclusion

Testing normal, boundary, and invalid cases helps identify incorrect peak-hour conditions and balance validation errors. Actual results should be compared against the expected values and recorded with execution evidence.